

vLLM源码解析3，块管理器BlockManager（上篇）

大家好，本篇我们进入“有趣的”（，反复告诉自己它很有趣，那么它一定能变得有趣起来）且“重要的”（这是真得很重要）的vllm块管理器相关代码解读。

vllm块管理器又分成朴素块管理器（UncachedBlockAllocator）和prefix caching型块管理器（CachedBlockAllocator）。本篇我们先讲比较简单的前者，下篇我们来细看更有趣也是更难的后者。

一、前情提要

在之前对调度器策略（Scheduler）的讲解中，我们主要说明了以下几点：

- 从vLLM批处理的入口函数开始，介绍了其推理内核LLMEngine的两个重要函数 `add_request()` 和 `step()`
- 在LLMEngine开始处理请求前（实例化阶段），它会先做一次模拟实验，来估计gpu上需要预留多少显存给KV Cache block。
- 当LLMEngine开始处理请求时(`add_request`)，它会把每个prompt当成一个请求，同时把它包装成一个SequenceGroup对象。
- 当LLMEngine开始执行1次调度时（`step`），调度器策略(Scheduler)会选择要送哪些seq_group去做新一轮推理。注意，在1次推理中，所有seq_group要么一起做prefill，要么一起做decode。

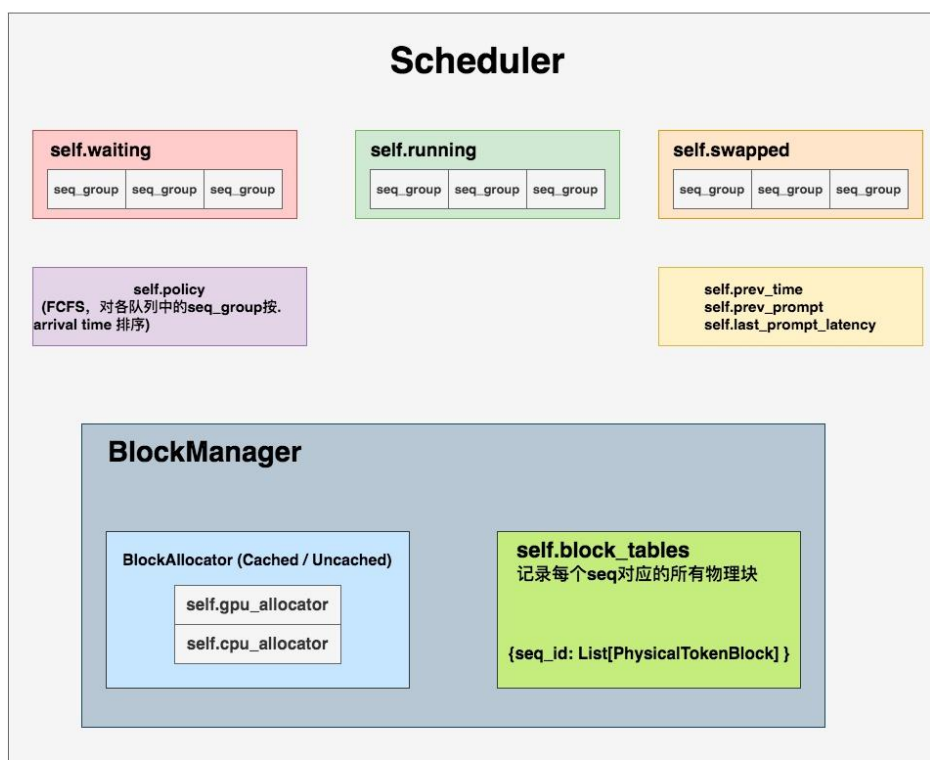
调度器策略流程图清晰版可参见这里：[猛猿：图解大模型计算加速系列：vLLM源码解析2，调度器策略\(Scheduler\)](#)

同时，我们遗留了以下问题：

- **问题1：vLLM的物理块管理（block manager）的细节**，包括物理块结构，逻辑块-物理块映射，物理块新增与释放，prefix caching等等
- **问题2：step()其余步骤**：调度器只是决定了要送哪些seq_group去做推理，但是“每1个推理阶段结束后，如何根据推理结果更新seq_group，并将其送入下一次调度”这块不是调度器的职责，这也是后面我们要讲解的“step()的其余步骤”。

今天我们就对问题1进行解答。问题2我们放在源码解读第四篇进行讲解

二、两种不同类型的BlockAllocator



知乎 @猛猿

在[源码解读2](#)中，我们画过Scheduler的架构图，它的下面维护着今天我们要细讲的块管理器（BlockManager），这也是vLLM自定义的一个class。

截止本文写作时，vLLM提供了BlockSpaceManagerv1和BlockSpaceManagerv2两个版本的块管理器。V1是vLLM默认的版本，V2是改进版本（但还没开发完，例如不支持prefix caching等功能）。所以本文依然基于BlockSpaceManagerv1进行讲解。

BlockManager这个class下又维护着两个重要属性：

- **BlockAllocator**：物理块分配者，负责实际为seq做物理块的分配、释放、拷贝等操作。其下又分成 `self.gpu_allocator` 和 `self.cpu_allocator` 两种类型，分别管理gpu和cpu上的物理块。
- **self.block_tables**：负责维护每个seq下的物理块列表，本质上它是一个字典，形式如 `{seq_id: List[PhysicalTokenBlock]}`。注意，这个字典维护着【所有】seq_group下seq的物理块，而不是单独某一个seq的。因为调度器是全局的，所以它下面的BlockManager自然也是全局的。

其中，BlockAllocator又分成两种类型：

- **CachedBlockAllocator**：按照prefix caching的思想来分配和管理物理块。在原理篇中，我们提过又些prompts中可能含有类似system message（例如，“假设你是一个能提供帮助的行车导航”）E等prefix信息，带有这些相同prefix信息的prompt完全可以共享用于存放prefix的物理块，这样既节省显存，也不用再对prefix做推理。

- `UncachedBlockAllocator`：正常分配和管理物理块，没有额外实现prefix caching的功能。

在块管理器的上篇中，我们介绍 `UncachedBlockAllocator`，在下篇中我们介绍更为复杂的 `CachedBlockAllocator`。

三、物理块和逻辑块结构

首先我们来快速回顾下在vllm中一个物理块和一个逻辑块长什么样子。

```
# vllm/block.py
class PhysicalTokenBlock:
    """Represents the state of a block in the KV cache."""

    def __init__(
        self,
        device: Device,
        block_number: int,
        block_size: int,
        block_hash: int,
        num_hashed_tokens: int,
    ) -> None:
        # =====
        # 设备, gpu/cpu
        # =====
        self.device = device
        # =====
        # 该物理块在对应设备上的全局block index
        # =====
        self.block_number = block_number
        # =====
        # 该物理块的尺寸（即槽位数量，默认为16）
        # =====
        self.block_size = block_size
        # =====
        # 该物理块的hash值
        # （在prefix caching场景下使用，非此场景则附值为-1）
        # =====
        self.block_hash = block_hash
        # =====
        # 该物理块的hash值是由多少个前置token计算而来的
        # （prefix caching场景下使用，非此场景则附值为0）
        # =====
        self.num_hashed_tokens = num_hashed_tokens
        # =====
        # 该物理块被多少个逻辑块引用
        # =====
        self.ref_count = 0
        # =====
        # 该物理块最后一次被使用的时间
        # （prefix caching场景下使用，非此场景则附值为-1）
        # =====
```

```

self.last_accessed = DEFAULT_LAST_ACCESSED_TIME

# =====
# 该物理块是否被计算过
# （prefix caching场景下使用）
# =====

self.computed = False

def __repr__(self) -> str:
    return (f'PhysicalTokenBlock(device={self.device}, '
            f'block_number={self.block_number}, '
            f'num_hashed_tokens={self.num_hashed_tokens}, '
            f'ref_count={self.ref_count}, '
            f'last_accessed={self.last_accessed}, '
            f'computed={self.computed})')

```

这里有一些和prefix caching相关的物理块属性，大家现在可能还看得一头雾水，不要担心，在块管理器的下篇中我们再来细讲，这里可以忽略。

逻辑块结构（一切尽在注释中）：

```

# # vllm/block.py
class LogicalTokenBlock:
    """A block that stores a contiguous chunk of tokens from left to right.

    Logical blocks are used to represent the states of the corresponding
    physical blocks in the KV cache.

    KV cache的逻辑块
    """

    def __init__(
        self,
        block_number: int, # 逻辑块的序号
        block_size: int, # 每个逻辑块中有多少个槽位（默认为16）
    ) -> None:
        self.block_number = block_number
        self.block_size = block_size

        # 逻辑块刚初始化时，将其中的每个token_id都初始化为_BLANK_TOKEN_ID (-1)
        self.token_ids = [_BLANK_TOKEN_ID] * block_size
        # 当前逻辑块中已经装下的token的数量
        self.num_tokens = 0

    def is_empty(self) -> bool:
        """判断当前逻辑块是否为空"""
        return self.num_tokens == 0

    def get_num_empty_slots(self) -> int:
        """当前逻辑块的空余槽位"""
        return self.block_size - self.num_tokens

    def is_full(self) -> bool:
        """判断当前逻辑块是否已经被装满"""

```

```

        return self.num_tokens == self.block_size

    def append_tokens(self, token_ids: List[int]) -> None:
        """将给定的一些token_ids装入当前逻辑块中"""
        # 给定的token_ids的长度必须 <= 当前逻辑块剩余的槽位
        assert len(token_ids) <= self.get_num_empty_slots()
        # 当前逻辑块第一个空槽的序号
        curr_idx = self.num_tokens
        # 将这些tokens装进去
        self.token_ids[curr_idx:curr_idx + len(token_ids)] = token_ids
        # 更新当前逻辑块中tokens的数量
        self.num_tokens += len(token_ids)

    def get_token_ids(self) -> List[int]:
        """获取当前逻辑块中所有被装满的位置的token_ids"""
        return self.token_ids[:self.num_tokens]

    def get_last_token_id(self) -> int:
        """获取当前逻辑块中所有被装满的位置的最后一个token_id"""
        assert self.num_tokens > 0
        return self.token_ids[self.num_tokens - 1]

```

关于逻辑块，我们已在源码解读2（TODO）的2.3（2）中详细介绍过，它是Sequence实例（seq）下维护的一个属性。我们也提过，在vLLM代码实现中：

- 每个seq维护自己的一份逻辑块列表，
- BlockManager中的self.block_tables（形式如：{seq_id: List[PhysicalBlock]}）则记录者每个seq下的物理块列表

通过seq这个中介，我们维护起“逻辑块->物理块”的映射

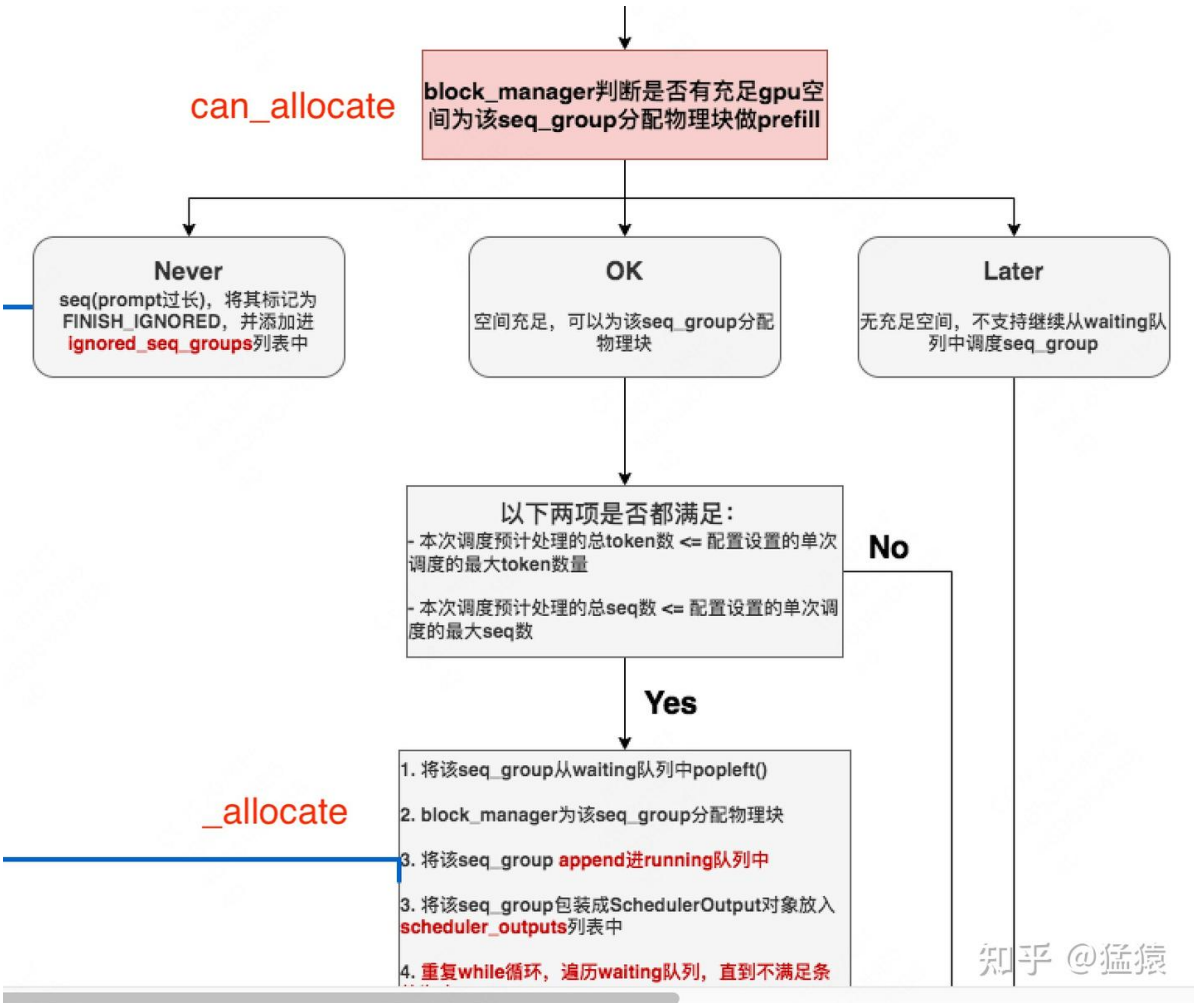
四、UncachedBlockAllocator

本文我们先来看较为简单的非缓存式BlockAllocator的实现。

4.1 在调度器中，什么时候会用到BlockAllocator

在调度器策略（TODO）的讲解中，我们明确了非常重要的一点：在vllm的1个推理阶段，所有的seq_group要么一起做prefill，要么一起做decode。这也意味着，某次调度的结果，要么全部来自waiting队列（等待做prefill的），要么全部来自running或者running + swapped队列（等待做decode的）。

4.2 为waiting队列中的seq_group分配prefill需要的物理块



如上图，当我们准备从waiting队列中调度seq_group时，我们会依次做两件事：

- 调用 `self.block_manager.can_allocate(seq_group)` 方法，判断当前gpu上是否有充足的空间，能为当下这`seq_group`的prefill阶段分配充足的物理块，用于装其KV Cache（细节我们在源码解读2中已讲过，这里不再赘述）
- 一旦我们认为当下空间充足，则调用 `self._allocate(seq_group)` 方法，为waiting队列中的这个`seq_group`实际分配物理块，这时我们会运用到`BlockAllocator`，并且`BlockAllocator`的类型不同（即是否做prefix caching），`allocate`的方法也会不同。

所以现在，我们就来看 `self._allocate(seq_group)` 函数（如何为waiting队列中的`seq_group`分配物理块做prefill）

`self._allocate(seq_group)` 的入口函数如下(一切尽在注释中)：

```
# vllm/core/scheduler.py
def _allocate(self, seq_group: SequenceGroup) -> None:
    # =====
    # block_manager为当前seq_group分配物理块
    # =====
    self.block_manager.allocate(seq_group)

    # =====
    # 当前seq_group状态改为running
    # =====
    for seq in seq_group.get_seqs(status=SequenceStatus.WAITING):
        seq.status = SequenceStatus.RUNNING
```

接下来我们看 `self.block_manager.allocate(seq_group)` 实现，如前文所说，本文我们解读的是 `BlockSpaceManagerV1`，所以我们就去这个class的顶一下看allocate方法（一切尽在注释中）。

```
# vllm/core/block_manager_v1.py
class BlockSpaceManagerV1(BlockSpaceManager):
    """Manages the mapping between logical and physical token blocks."""

    def __init__(
        self,
        block_size: int, # 每个block的槽位大小，默认为16
        num_gpu_blocks: int, # 当前gpu上最多能分配的block数量
        num_cpu_blocks: int, # 当前cpu上，用于做swap的内存中，最多能分配的block数量
        watermark: float = 0.01, # 内存交换的水位线（阈值）
        sliding_window: Optional[int] = None, # 滑动窗口的大小
        enable_caching: bool = False, # 是否需要做prefix caching
    ) -> None:

        self.block_size = block_size
        self.num_total_gpu_blocks = num_gpu_blocks
        self.num_total_cpu_blocks = num_cpu_blocks

        if enable_caching and sliding_window is not None:
            raise NotImplementedError(
                "Sliding window is not allowed with prefix caching enabled!")

        self.block_sliding_window = None
        if sliding_window is not None:
            assert sliding_window % block_size == 0, (sliding_window,
                                                        block_size)
            self.block_sliding_window = sliding_window // block_size

        self.watermark = watermark
        assert watermark >= 0.0

        self.enable_caching = enable_caching

        #
        =====

        # 水位线block数量：理解成一个阈值，这个阈值决定是否要给当前seq分配block
```

```

# 设置水位线block的目的是不要一下打满设备中的物理块，留一些buffer，避免后续频繁地发生
swap

#
=====

self.watermark_blocks = int(watermark * num_gpu_blocks)

#
=====

# 根据是否做了prefix caching限制，来选择不同的allocator
#
=====

if self.enable_caching:
    logger.info("Automatic prefix caching is enabled.")
    self.gpu_allocator = CachedBlockAllocator(Device.GPU, block_size,
                                                num_gpu_blocks)
    self.cpu_allocator = CachedBlockAllocator(Device.CPU, block_size,
                                                num_cpu_blocks)
else:
    self.gpu_allocator = UncachedBlockAllocator(
        Device.GPU, block_size, num_gpu_blocks)
    self.cpu_allocator = UncachedBlockAllocator(
        Device.CPU, block_size, num_cpu_blocks)

#
=====

# 创建block_tables字典，形式如{seq_id: block_table}，记录每一个序列对应的block
table
#
=====

self.block_tables: Dict[int, BlockTable] = {}

def can_allocate(self, seq_group: SequenceGroup) -> AllocStatus:
    """
    确实是否可以给这个seq_group分配物理块，返回结果有三种情况：
    - AllocStatus.NEVER: 不分配；
    - AllocStatus.OK: 可以分配；
    - AllocStatus.LATER: 延迟分配
    在源码解读2中我们详细讲过这个方法，这里不赘述
    """
    ...

def allocate(self, seq_group: SequenceGroup) -> None:
    """
    为当前seq_group分配物理块做prefill
    """
    #
    =====

    # NOTE: vllm中有一条重要假设：一个seq_group内的所有seq都共享一个prompt
    # 而我们现在正是要对这个prompt分配物理块。
    # 复习一下，waiting队列中所有的seq_group都没做过prefill，因此每个seq_group下面
    # 只有1条seq，这个seq即位prompt本身，所以我们取[0]即可拿出这个prompt
    #
    =====

    seq = seq_group.get_seqs(status=SequenceStatus.WAITING)[0]

```

```

#
=====

# 计算该seq的逻辑块数量
# （prefill阶段，有多少个逻辑块，就应该分配多少个物理块）
#
=====

num_prompt_blocks = len(seq.logical_token_blocks)

#
=====

# 为该seq分配物理块，List[PhysicalTokenBlock]
#
=====

block_table: BlockTable = []
# 遍历该seq的所有逻辑块
for logical_idx in range(num_prompt_blocks):
    #
=====

    # 如果block的滑动窗口长度不为空（可暂时忽略不看）
    #
=====

    if (self.block_sliding_window is not None
        and logical_idx >= self.block_sliding_window):
        block = block_table[logical_idx % self.block_sliding_window]
        # Set the reference counts of the token blocks.
        block.ref_count = seq_group.num_seqs()

    #
=====

    # 如果做了prefix caching，即使用的是CachedBlockAllocator
    # （是下篇要讲解的重点，这里我们用的是UncachedBlockAllocator，所以可忽略不看）
    #
=====

    elif self.enable_caching:
        block = self.gpu_allocator.allocate(
            seq.hash_of_block(logical_idx),
            seq.num_hashed_tokens_of_block(logical_idx))

    #
=====

    # 其余情况（即UncachedBlockAllocator对应的情况）
    #
=====

    else:
        # 从空闲物理块中取一块出来，并令其ref_count = 1（表示有1个逻辑块引用它了）
        # 相关代码讲解见下
        block = self.gpu_allocator.allocate()
        # 由于seq_group下的所有seq共享一个prompt，
        # 所以进一步令物理块的ref_count = num_seqs
        # （表示这些seqs的逻辑块都引用它了）
        block.ref_count = seq_group.num_seqs()

        block_table.append(block)

#
=====

```

```

# prefill阶段，这个seq_group下所有的seq共享一个prompt，也即共享这个prompt代表的物理块
#
=====
for seq in seq_group.get_seqs(status=SequenceStatus.WAITING):
    self.block_tables[seq.seq_id] = block_table.copy()

# ... (该class下的其它方法，暂时略过)

```

那现在我们再进一步看下上面代码中 `block = self.gpu_allocator.allocate()` 的实现（一切尽在注释中）：

```

# vllm/core/block_manager_v1.py
class UncachedBlockAllocator(BlockAllocatorBase):
    """Manages free physical token blocks for a device.

    The allocator maintains a list of free blocks and allocates a block when
    requested. When a block is freed, its reference count is decremented. If
    the reference count becomes zero, the block is added back to the free list.
    """

    def __init__(
        self,
        device: Device,
        block_size: int,
        num_blocks: int,
    ) -> None:
        self.device = device # 设备: cpu/gpu
        self.block_size = block_size # 该设备上每个物理块的槽位数，默认为16
        self.num_blocks = num_blocks # 该设备上留给KV cache的总物理块数量

        # =====
        # 初始化所有物理块
        # self.free_blocks: List[PhysicalTokenBlock]，用于跟踪该设备上所有
        # 未被使用过的物理块
        # =====
        self.free_blocks: BlockTable = []
        for i in range(num_blocks):
            # vllm/vllm/block.py
            # 定义物理块
            block = PhysicalTokenBlock(device=device,
                                       block_number=i,
                                       block_size=block_size,
                                       block_hash=-1,
                                       num_hashed_tokens=0)
            self.free_blocks.append(block)

    def allocate(self,
                 block_hash: Optional[int] = None,
                 num_hashed_tokens: int = 0) -> PhysicalTokenBlock:
        if not self.free_blocks:

```

```

        raise ValueError("Out of memory! No free blocks are available.")
    block = self.free_blocks.pop()
    block.ref_count = 1 # 该物理块首次有逻辑块引用了，所以ref_count=1
    return block

def free(self, block: PhysicalTokenBlock) -> None:
    """
    释放一条seq对应的物理块
    即将对应物理块的引用-1，如果此时引用数量为0，说明对应物理块完全自由了，
    需要再将其放入自由物理块列表中
    """
    if block.ref_count == 0:
        raise ValueError(f"Double free! {block} is already freed.")
    block.ref_count -= 1
    if block.ref_count == 0:
        self.free_blocks.append(block)

def get_num_free_blocks(self) -> int:
    return len(self.free_blocks)

def contains_block(self, block_hash: int) -> bool:
    raise NotImplementedError(
        "Invalid codepath for uncached block allocator.")

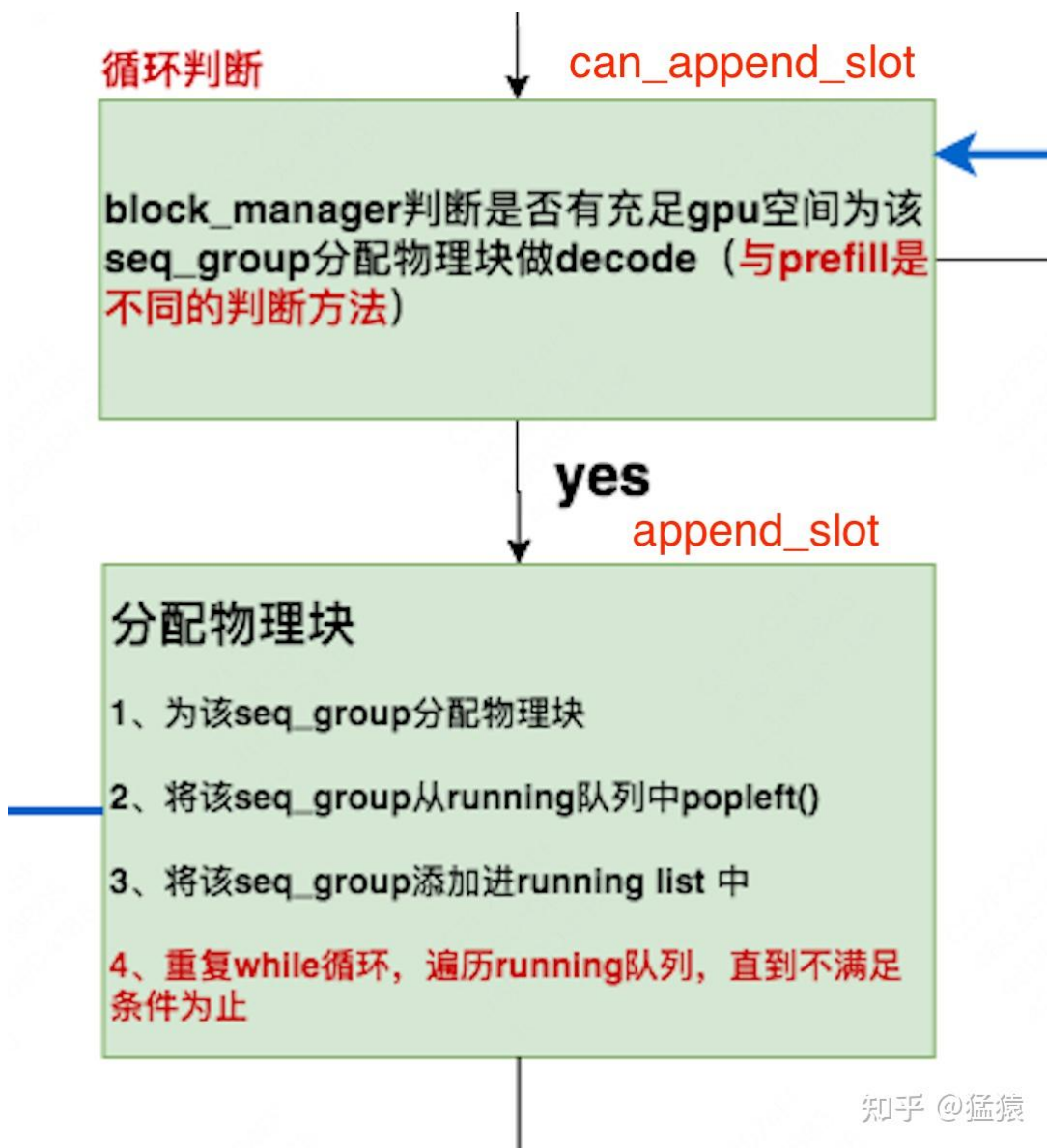
def update_hash(self, block_hash: int, block: PhysicalTokenBlock):
    raise NotImplementedError(
        "Invalid codepath for uncached block allocator.")

```

好，整个过程代码注释已经说得非常清楚了，这里我们再稍微总结下：

- waiting队列中的每个seq_group都还未经历过prefill阶段，因此每个seq_group下只有1个seq，这个seq即为prompt
- 在使用 `UncachedBlockAllocator` 为waiting队列中的某个seq_group分配物理块时，我们就是在对初始的这个prompt分配物理块。所以这个prompt有多少个逻辑块，我们就分配多少个可用的空闲物理块，同时注意更新物理块的ref_count。
- 你一定发现了，这里我们做的只是**给定一种“物理块的分配方案”**，我们只是在制定这个seq_group可以使用哪些物理块，**但并没有实际往物理块中添加数据！“添加数据”这一步留到这1步推理实际开始时，由CacheEngine按照这个方案，往物理块中实际添加KV Cache。这个我们留在再后面的系列讲解。**

4.3 为running/swapped队列中的seq_group分配decode需要的物理块



接下来我们考虑为running/swapped队列中的seq_group分配decode需要的物理块。

对于每个seq_group，在上1个推理阶段，我们对它下面的每个seq都产出了1个token。所以在这个推理阶段，我们判断能否为这些seq_group分配物理块时，我们也会分成两步：

- 调用 `self.block_manager.can_append_slot(seq_group)` 方法，判断是否至少能为这个seq_group下的每个seq都分配1个空闲物理块。如果可以则认为能调度这个seq_group（原因和代码分析我们在源码解读2中细讲过，这里不赘述）。
- 调用 `self._append_slot(seq_group, blocks_to_copy)` 方法，实际分配物理块。我们马上来看细节。

调用入口（一切尽在注释中）：

```
# vllm/core/scheduler.py
def _append_slot(
    self,
    seq_group: SequenceGroup,
    blocks_to_copy: Dict[int, List[int]], # {旧物理块id: [由旧物理块copy-on-
write而来的新物理块id]}
) -> None:
    #
    =====
```

```

# 遍历这个seq_group中状态为running的所有seq
#
=====
for seq in seq_group.get_seqs(status=SequenceStatus.RUNNING):
    #
=====
    # 为这个seq分配物理块，代码细节见下
    # ret = None时，说明可以继续使用物理块的空槽位，不需要新分配物理块
    # ret不为空时的结果为：(last_block.block_number,
new_block.block_number)
    # 前者表示源物理块，后者表示copy-on-write而来的物理块
    # （有疑惑不要紧，下文我们马上来查看代码细节）
    #
=====
    ret = self.block_manager.append_slot(seq)
    #
=====
    # ret非None，说明采用了copy-on-write机制（参见原理篇讲解）
    # 这时我们要记录copy-on-write相关的映射关系
    #
=====
    if ret is not None:
        src_block, dst_block = ret
        # {旧物理块id: [由旧物理块copy-on-write而来的新物理块id]}
        if src_block in blocks_to_copy:
            blocks_to_copy[src_block].append(dst_block)
        else:
            blocks_to_copy[src_block] = [dst_block]

```

我们来看 `self.block_manager.append_slot(seq)` 细节（一切尽在注释中）：

```

# vllm/core/block_manager_v1.py
class BlockSpaceManagerV1(BlockSpaceManager):
    """Manages the mapping between logical and physical token blocks."""

    def __init__(
        self,
        block_size: int, # 每个block的大小
        num_gpu_blocks: int, # 当前gpu上最多能分配的block数量
        num_cpu_blocks: int, # 当前cpu上，用于做swap的内存中，最多能分配的block数量
        watermark: float = 0.01, # 内存交换的水位线（阈值）
        sliding_window: Optional[int] = None, # 滑动窗口的大小
        enable_caching: bool = False, # 是否需要做prefix caching（目前暂时不支持，所以
都设为False）
    ) -> None:

        self.block_size = block_size
        self.num_total_gpu_blocks = num_gpu_blocks
        self.num_total_cpu_blocks = num_cpu_blocks

        if enable_caching and sliding_window is not None:
            raise NotImplementedError(
                "sliding window is not allowed with prefix caching enabled!")

```

```

self.block_sliding_window = None
if sliding_window is not None:
    assert sliding_window % block_size == 0, (sliding_window,
                                              block_size)
    self.block_sliding_window = sliding_window // block_size

self.watermark = watermark
assert watermark >= 0.0

self.enable_caching = enable_caching

#
=====
# 水位线block数量：理解成一个阈值，这个阈值决定是否要给当前seq分配block
# 设置水位线block的目的是不要一下打满设备中的物理块，留一些buffer，避免后续频繁地发生
swap
#
=====
self.watermark_blocks = int(watermark * num_gpu_blocks)

#
=====
# 根据是否做了prefix caching限制，来选择不同的allocator
#
=====
if self.enable_caching:
    logger.info("Automatic prefix caching is enabled.")
    self.gpu_allocator = CachedBlockAllocator(Device.GPU, block_size,
                                              num_gpu_blocks)
    self.cpu_allocator = CachedBlockAllocator(Device.CPU, block_size,
                                              num_cpu_blocks)
else:
    self.gpu_allocator = UncachedBlockAllocator(
        Device.GPU, block_size, num_gpu_blocks)
    self.cpu_allocator = UncachedBlockAllocator(
        Device.CPU, block_size, num_cpu_blocks)

#
=====
# 创建block_tables字典，形式如{seq_id: block_table}，记录每一个序列对应的block
table
#
=====
self.block_tables: Dict[int, BlockTable] = {}

def can_append_slot(self, seq_group: SequenceGroup) -> bool:
    """
    对于这个seq_group，我们检查对于其中的每一个seq，
    是否能至少分配一个空闲物理块给它
    相关讲解在源码解读2中详细说过，不再赘述
    """
    # Simple heuristic: If there is at least one free block
    # for each sequence, we can append.
    num_free_gpu_blocks = self.gpu_allocator.get_num_free_blocks()
    num_seqs = seq_group.num_seqs(status=SequenceStatus.RUNNING)

```

```

        return num_seqs <= num_free_gpu_blocks

def _promote_last_block(
    self,
    seq: Sequence,
    last_block: PhysicalTokenBlock,
) -> PhysicalTokenBlock:
    assert self.enable_caching

    # Compute a new hash for the block so that it can be shared by other
    # Sequences
    new_hash = seq.hash_of_block(len(seq.logical_token_blocks) - 1)

    # if new_hash is already in the cached table, then free last_block
    # and return the cached version
    if self.gpu_allocator.contains_block(new_hash):
        self.gpu_allocator.free(last_block)
        return self.gpu_allocator.allocate(new_hash)
    else:
        self.gpu_allocator.update_hash(new_hash, last_block)
        return last_block

def _is_last_block_full(
    self,
    seq: Sequence,
) -> bool:
    """
    检查当前这最后一个物理块是不是已经装满了
    """

    # 获取该seq的token数量
    token_ids_len = len(seq.data.get_token_ids())
    # 如果seq的token数量大于0, 且token数量能被block整除, 说明当前这最后一个物理块是满的
    return token_ids_len > 0 and token_ids_len % seq.block_size == 0

def _maybe_promote_last_block(
    self,
    seq: Sequence,
    last_block: PhysicalTokenBlock,
) -> PhysicalTokenBlock:
    # =====
    # 检查当前这最后一个物理块是否满了, 如果是:
    # =====
    if self._is_last_block_full(seq):
        return self._promote_last_block(seq, last_block)
    else:
        return last_block

def _allocate_last_physical_block(
    self,
    seq: Sequence,
) -> PhysicalTokenBlock:
    """
    我们在想添加新的物理块之前, 调用这个函数, 来判断是不是真得有必要添加一个物理块
    """

    # =====

```

```

# 如果不使用prefix caching, 就直接分配物理块 (看到这里就可以, 下面的不用看)
# =====
if not self.enable_caching:
    return self.gpu_allocator.allocate()

# =====
# 使用prefix caching (下篇要讲解的重点, 这里可以忽略)
# =====
block_hash: Optional[int] = None
if (self._is_last_block_full(seq)):
    block_hash = seq.hash_of_block(len(seq.logical_token_blocks) - 1)
num_hashed_tokens = seq.num_hashed_tokens_of_block(
    len(seq.logical_token_blocks) - 1)

new_block = self.gpu_allocator.allocate(block_hash, num_hashed_tokens)

if block_hash is None:
    assert new_block.ref_count == 1
return new_block

def append_slot(
    self,
    seq: Sequence,
) -> Optional[Tuple[int, int]]:
    """
    为这个seq中的新token分配一个物理槽位
    """
    # =====
    # 读取这个seq的逻辑块, List[LogicalTokenBlock]
    # =====
    logical_blocks = seq.logical_token_blocks
    # =====
    # 读取这个seq的物理块, List[PhysicalTokenBlock]
    # =====
    block_table = self.block_tables[seq.seq_id]

    # =====
    # 如果物理块数量 < 逻辑块数量 (说明此时需要分配新的物理块了)
    # 注: 上1个推理阶段完毕后, seq的逻辑块更新了 (把最新生成的这个token装进去了)
    # 但物理块还没更新
    # =====
    if len(block_table) < len(logical_blocks):
        # =====
        # (需要声明物理块只允许比逻辑块少1块)
        # =====
        assert len(block_table) == len(logical_blocks) - 1

        # =====
        # 如果使用滑动窗口时的逻辑 (可暂时忽略不看)
        # =====
        if (self.block_sliding_window
            and len(block_table) >= self.block_sliding_window):
            # reuse a block
            block_table.append(block_table[len(block_table) %
                                         self.block_sliding_window])

        # =====

```

```

# 其余情况，直接分配一个新的物理块给当前序列
# =====
else:
    # 如果是UncachedBlockAllocator，就直接分配一个新的空闲物理块
    new_block = self._allocate_last_physical_block(seq)
    block_table.append(new_block)
    return None

# =====
# 如果物理块数量==逻辑块数量:
# =====
last_block = block_table[-1] # 取出最后一个物理块
assert last_block.device == Device.GPU # 声明必须是gpu物理块

# =====
# 如果最后一个物理块的引用数量为1（只有1个逻辑块引用它）
# （也就是只有当前这个seq在用它）
# =====
if last_block.ref_count == 1:
    # =====
    # 如果你是在做prefix caching（暂时不看，下篇再细讲）
    # =====
    if self.enable_caching:
        maybe_new_block = self._maybe_promote_last_block(
            seq, last_block)
        block_table[-1] = maybe_new_block

    # =====
    # 不用prefix caching，此时我们不需要添加新的物理块，所以返回None
    # =====
    return None

# =====
# 如果最后一个物理块的引用数量为 > 1（有别的逻辑块在引用它）
# （也就是有别的seq在用它）
# =====
else:
    # =====
    # The last block is shared with other sequences.
    # Copy on Write: Allocate a new block and copy the tokens.
    # 触发copy-on-write机制，分配一个新的物理块。机制相关的解释见原理篇讲解
    # =====
    new_block = self._allocate_last_physical_block(seq)
    block_table[-1] = new_block
    # 从该seq的block_table中释放掉旧的物理块
    # 也即该物理块ref_count -= 1，如果-=1后ref_count=0，说明该物理块彻底自由了，
    # 此时可以把它添加进自由物理块的列表中（细节留给大家自己看源码，不难）
    self.gpu_allocator.free(last_block)

    return last_block.block_number, new_block.block_number

```

如果在阅读上述代码中，你感觉有些迷惑，建议先看一下原理篇的copy-on-write机制，以及源码解读2中的相关相关讲解。动手画画图，帮助理清过程。

同样，在这里我们依然要强调，调度器中只是给出了物理块的分配方案，并没有实际往物理块中添加数据，添加数据这一步是CacheEngine照着这个方案来实际操作的，这个我们放在后面的文章中讲解。

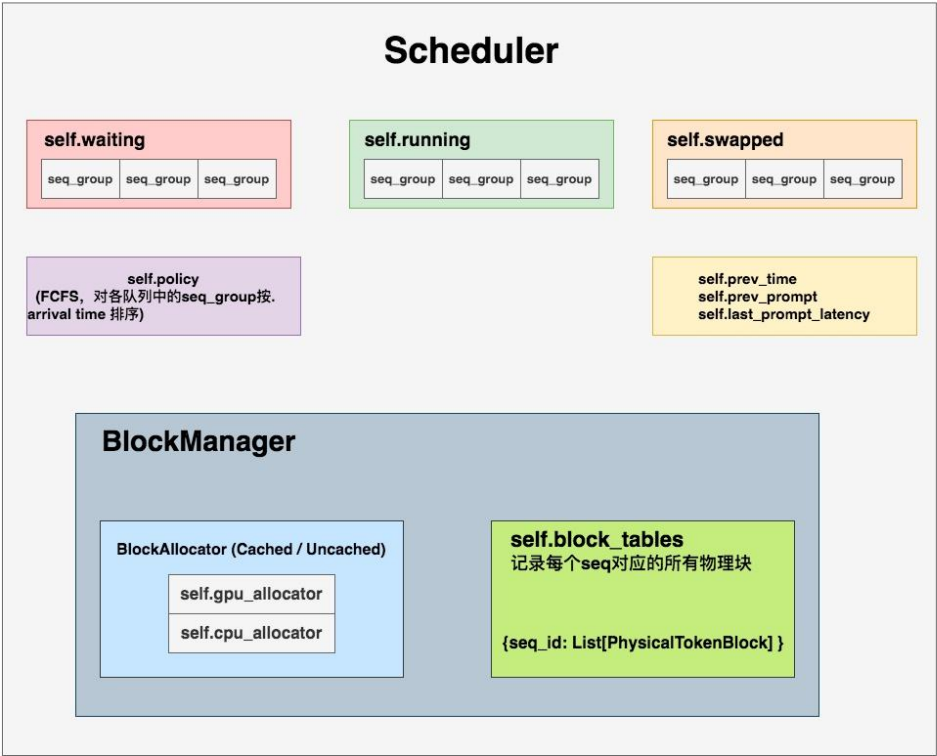
恭喜你已经了解了非缓存式物理块管理器(`UncachedBlockAllocator`)的全部细节！在块管理器的下篇中，我们将来看一个更有意思，代码上也更有难度的缓存式块管理器 `CachedBlockAllocator`，一起来看看vllm在论文中说的prefix caching是如何实现的。

vLLM源码解析3，Prefix Caching

大家好，今天我们继续vllm源码的解析，一起来看下它最近总是被频繁提起、也是较不好理解的一个创新点：Prefix Caching（本文同时也是BlockManager的下篇，虽然标题没有提及）

说一些题外话，之前写vllm源码解读的文章，阅读量不是很高，再加上写这类型文章真得耗时耗力耗头发（自己看懂代码容易，但是给别人讲懂很难，把代码转变成一篇有逻辑的、兼顾全局和细节的文章就更难了。特别是mlsys的代码，懂的都懂），因此我一度丧失了对它的写作热情。但是这段日子打开尘封已久的私信，竟然看到有很多朋友在催更，所以动力又回来了些。不过这个系列后续的更新节奏依然还是比较慢（我的大部分文章都是在午休时间见缝插针写的），还请大家见谅哈。

一、两种不同类型的BlockAllocator



知乎 @猛猿

在源码解读2中，我们画过Schduler的架构图，它的下面维护着今天我们要细讲的块管理器（BlockManager），这也是vLLM自定义的一个class。

截止本文开始写作时，vLLM提供了 `BlockSpaceManagerV1` 和 `BlockSpaceManagerV2` 两个版本的块管理器。V1是vLLM默认的版本，V2是改进版本（但还没开发完，例如不支持prefix caching等功能）。所以本文依然基于 `BlockSpaceManagerV1` 进行讲解。

`BlockManager`这个class下又维护着两个重要属性：

- `BlockAllocator`：物理块分配者，负责实际为seq做物理块的分配、释放、拷贝等操作。其下又分成 `self.gpu_allocator` 和 `self.cpu_allocator` 两种类型，分别管理gpu和cpu上的物理块。
- `self.block_tables`：负责维护每个seq下的物理块列表，本质上它是一个字典，形式如 `{seq_id: List[PhysicalTokenBlock]}`。注意，这个字典维护着【所有】seq_group下seq的物理块，而不是单独某一个seq的。因为调度器是全局的，所以它下面的的`BlockManager`自然也是全局的。

其中，`BlockAllocator`又分成两种类型：

- `CachedBlockAllocator`：按照prefix caching的思想来分配和管理物理块，是本篇讲解的重点。在原理篇中，我们提过：
 - 在prefill阶段，prompts中可能含有类似system message（例如，“假设你是一个能提供帮助的行车导航”）等prefix信息，带有这些相同prefix信息的prompt完全可以共享物理块，**实现节省显存、减少重复计算的目的**。
 - 在decode阶段，我们依然可以用这种prefix的思想，及时发现可以重复利用的物理块。
 - prefill和decode阶段做prefix caching的方法有些不同，我们会在后文仔细讲解。
- `UncachedBlockAllocator`：正常分配和管理物理块，没有额外实现prefix caching的功能。这是我们源码解读3讲解的重点，本文不再赘述。

二、物理块和逻辑块结构

首先我们来快速回顾下在vllm中一个物理块和一个逻辑块长什么样子。

2.1 物理块结构

一切尽在注释中：

```
# vllm/block.py
class PhysicalTokenBlock:
    """Represents the state of a block in the KV cache."""

    def __init__(
        self,
        device: Device,
```

```

        block_number: int,
        block_size: int,
        block_hash: int,
        num_hashed_tokens: int,
    ) -> None:
        # =====
        # 设备, gpu/cpu
        # =====
        self.device = device
        # =====
        # 该物理块在对应设备上的全局block index
        # =====
        self.block_number = block_number
        # =====
        # 该物理块的尺寸(即槽位数量, 默认为16)
        # =====
        self.block_size = block_size
        # =====
        # 该物理块的hash值
        # (在prefix caching场景下使用, 非此场景则附值为-1)
        # =====
        self.block_hash = block_hash
        # =====
        # 该物理块的hash值是由多少个前置token计算而来的
        # (prefix caching场景下使用, 非此场景则附值为0)
        # =====
        self.num_hashed_tokens = num_hashed_tokens
        # =====
        # 该物理块被多少个逻辑块引用
        # =====
        self.ref_count = 0
        # =====
        # 该物理块最后一次被使用的时间
        # (prefix caching场景下使用, 非此场景则附值为-1)
        # =====
        self.last_accessed = DEFAULT_LAST_ACCESSED_TIME
        # =====
        # 该物理块是否被计算过
        # (prefix caching场景下使用)
        # =====
        self.computed = False

    def __repr__(self) -> str:
        return (f'PhysicalTokenBlock(device={self.device}, '
                f'block_number={self.block_number}, '
                f'num_hashed_tokens={self.num_hashed_tokens}, '
                f'ref_count={self.ref_count}, '
                f'last_accessed={self.last_accessed}, '
                f'computed={self.computed})')

```

2.2 逻辑块结构

一切尽在注释中：

```
# vllm/block.py
class LogicalTokenBlock:
    """A block that stores a contiguous chunk of tokens from left to right.

    Logical blocks are used to represent the states of the corresponding
    physical blocks in the KV cache.

    KV cache的逻辑块
    """

    def __init__(
        self,
        block_number: int, # 逻辑块的序号
        block_size: int, # 每个逻辑块中有多少个槽位（默认为16）
    ) -> None:
        self.block_number = block_number
        self.block_size = block_size

        # 逻辑块刚初始化时，将其中的每个token_id都初始化为_BLANK_TOKEN_ID (-1)
        self.token_ids = [_BLANK_TOKEN_ID] * block_size
        # 当前逻辑块中已经装下的token的数量
        self.num_tokens = 0

    def is_empty(self) -> bool:
        """判断当前逻辑块是为空"""
        return self.num_tokens == 0

    def get_num_empty_slots(self) -> int:
        """当前逻辑块的空余槽位"""
        return self.block_size - self.num_tokens

    def is_full(self) -> bool:
        """判断当前逻辑块是否已经被装满"""
        return self.num_tokens == self.block_size

    def append_tokens(self, token_ids: List[int]) -> None:
        """将给定的一些token_ids装入当前逻辑块中"""
        # 给定的token_ids的长度必须 <= 当前逻辑块剩余的槽位
        assert len(token_ids) <= self.get_num_empty_slots()
        # 当前逻辑块第一个空槽的序号
        curr_idx = self.num_tokens
        # 将这些tokens装进去
        self.token_ids[curr_idx:curr_idx + len(token_ids)] = token_ids
        # 更新当前逻辑块中tokens的数量
        self.num_tokens += len(token_ids)

    def get_token_ids(self) -> List[int]:
        """获取当前逻辑块中所有被装满的位置的token_ids"""
        return self.token_ids[:self.num_tokens]
```

```
def get_last_token_id(self) -> int:
    """获取当前逻辑块所有被装满的位置的最后一个token_id"""
    assert self.num_tokens > 0
    return self.token_ids[self.num_tokens - 1]
```

关于逻辑块，我们已在源码解读2的2.3（2）中详细介绍过，它是Sequence实例（seq）下维护的一个属性。我们也提过，在vLLM代码实现中：

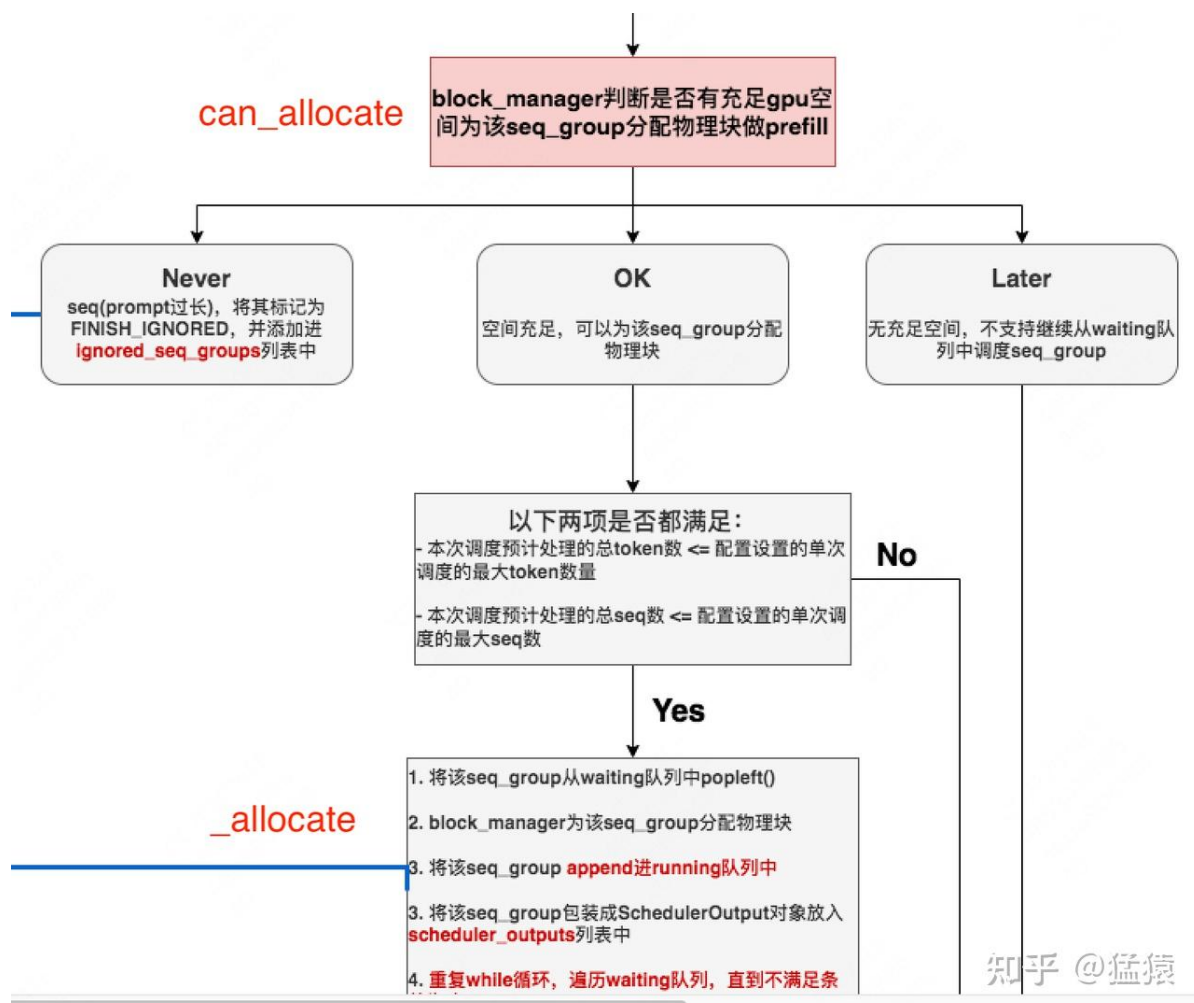
- 每个seq维护自己的一份逻辑块列表，
- BlockManager中的self.block_tables（形式如：{seq_id: List[PhysicalBlock]}）则记录者每个seq下的物理块列表

通过seq这个中介，我们维护起“逻辑块->物理块”的映射。

三、prefill阶段的物理块分配方法

在本节中，我们详细解读“如何使用CachedBlockAllocator为waiting队列中的seq_group分配做prefill需要的物理块”

3.1 allocate函数入口



如上图，当我们准备从waiting队列中调度seq_group时，我们会依次做两件事：

- 调用 `self.block_manager.can_allocate(seq_group)` 方法，判断当前gpu上是否有充足的空间，能为当下这seq_group的prefill阶段分配充足的物理块，用于装其KV Cache（细节我们在源码解读2中已讲过，这里不再赘述）

- 一旦我们认为当下空间充足，则调用 `self._allocate(seq_group)` 方法，为waiting队列中的这个seq_group实际分配物理块，这时我们会运用到BlockAllocator，并且BlockAllocator的类型不同（即是否做prefix caching），allocate的方法也会不同。

所以现在，我们就来看 `self._allocate(seq_group)` 函数（如何为waiting队列中的seq_group分配物理块做prefill）

`self._allocate(seq_group)` 的入口函数如下(一切尽在注释中):

```
# vllm/core/scheduler.py
def _allocate(self, seq_group: SequenceGroup) -> None:
    # =====
    # block_manager为当前seq_group分配物理块
    # =====
    self.block_manager.allocate(seq_group)

    # =====
    # 当前seq_group状态改为running
    # =====
    for seq in seq_group.get_seqs(status=SequenceStatus.WAITING):
        seq.status = SequenceStatus.RUNNING
```

接下来我们看 `self.block_manager.allocate(seq_group)` 实现，如前文所说，本文我们解读的是 `BlockSpaceManagerV1`，所以我们就去这个class的顶一下看allocate方法（一切尽在注释中）。

```
# vllm/core/block_manager_v1.py
class BlockSpaceManagerV1(BlockSpaceManager):
    """Manages the mapping between logical and physical token blocks."""

    def __init__(
        self,
        block_size: int, # 每个block的槽位大小，默认为16
        num_gpu_blocks: int, # 当前gpu上最多能分配的block数量
        num_cpu_blocks: int, # 当前cpu上，用于做swap的内存中，最多能分配的block数量
        watermark: float = 0.01, # 内存交换的水位线（阈值）
        sliding_window: Optional[int] = None, # 滑动窗口的大小
        enable_caching: bool = False, # 是否需要做prefix caching
    ) -> None:

        self.block_size = block_size
        self.num_total_gpu_blocks = num_gpu_blocks
        self.num_total_cpu_blocks = num_cpu_blocks

        if enable_caching and sliding_window is not None:
            raise NotImplementedError(
                "Sliding window is not allowed with prefix caching enabled!")

        self.block_sliding_window = None
        if sliding_window is not None:
            assert sliding_window % block_size == 0, (sliding_window,
                                                        block_size)
            self.block_sliding_window = sliding_window // block_size
```

```

self.watermark = watermark
assert watermark >= 0.0

self.enable_caching = enable_caching

#
=====
# 水位线block数量: 理解成一个阈值, 这个阈值决定是否要给当前seq分配block
# 设置水位线block的目的是不要一下打满设备中的物理块, 留一些buffer, 避免后续频繁地发生
swap
#
=====
self.watermark_blocks = int(watermark * num_gpu_blocks)

#
=====
# 根据是否做了prefix caching限制, 来选择不同的allocator
#
=====
if self.enable_caching:
    logger.info("Automatic prefix caching is enabled.")
    self.gpu_allocator = CachedBlockAllocator(Device.GPU, block_size,
                                                num_gpu_blocks)
    self.cpu_allocator = CachedBlockAllocator(Device.CPU, block_size,
                                                num_cpu_blocks)
else:
    self.gpu_allocator = UncachedBlockAllocator(
        Device.GPU, block_size, num_gpu_blocks)
    self.cpu_allocator = UncachedBlockAllocator(
        Device.CPU, block_size, num_cpu_blocks)

#
=====
# 创建block_tables字典, 形式如{seq_id: block_table}, 记录每一个序列对应的block
table
#
=====
self.block_tables: Dict[int, BlockTable] = {}

def can_allocate(self, seq_group: SequenceGroup) -> AllocStatus:
    """
    确实是否可以给这个seq_group分配物理块, 返回结果有三种情况:
    - AllocStatus.NEVER: 不分配;
    - AllocStatus.OK: 可以分配;
    - AllocStatus.LATER: 延迟分配
    在源码解读2中我们详细讲过这个方法, 这里不赘述
    """
    ...

def allocate(self, seq_group: SequenceGroup) -> None:
    """
    为当前seq_group分配物理块做prefill
    """

```

```

#
=====

# NOTE: vllm中有一条重要假设：一个seq_group内的所有seq都共享一个prompt
# 而我们现在正是要对这个prompt分配物理块。
# 复习一下，waiting队列中所有的seq_group都没做过prefill，因此每个seq_group下面
# 只有1条seq，这个seq即位prompt本身，所以我们取[0]即可拿出这个prompt
#
=====

seq = seq_group.get_seqs(status=SequenceStatus.WAITING)[0]

#
=====

# 计算该seq的逻辑块数量
# （prefill阶段，有多少个逻辑块，就应该分配多少个物理块）
#
=====

num_prompt_blocks = len(seq.logical_token_blocks)

#
=====

# 为该seq分配物理块，List[PhysicalTokenBlock]
#
=====

block_table: BlockTable = []
# 遍历该seq的所有逻辑块
for logical_idx in range(num_prompt_blocks):
    #
    =====

    # 如果block的滑动窗口长度不为空（可暂时忽略不看）
    #
    =====

    if (self.block_sliding_window is not None
        and logical_idx >= self.block_sliding_window):
        block = block_table[logical_idx % self.block_sliding_window]
        # Set the reference counts of the token blocks.
        block.ref_count = seq_group.num_seqs()

    #
    =====

    # 如果做了prefix caching，即使用的是CachedBlockAllocator
    # （本文重点，我们马上来讲解）
    #
    =====

    elif self.enable_caching:
        block = self.gpu_allocator.allocate(
            seq.hash_of_block(logical_idx),
            seq.num_hashed_tokens_of_block(logical_idx))

    #
    =====

    # 其余情况（即UncachedBlockAllocator对应的情况，非本文关注重点）
    #
    =====

    else:
        # 从空闲物理块中取一块出来，并令其ref_count = 1（表示有1个逻辑块引用它了）
        # 相关代码讲解见下
        block = self.gpu_allocator.allocate()

```

```

        # 由于seq_group下的所有seq共享一个prompt，
        # 所以进一步令物理块的ref_count = num_seqs
        # （表示这些seqs的逻辑块都引用它了）
        block.ref_count = seq_group.num_seqs()

        block_table.append(block)

    #
=====
    # prefill阶段，这个seq_group下所有的seq共享一个prompt，也即共享这个prompt代表的物理块
    #
=====
    for seq in seq_group.get_seqs(status=SequenceStatus.WAITING):
        self.block_tables[seq.seq_id] = block_table.copy()

    # ... (该class下的其它方法，暂时略过)

```

3.2 prefill阶段计算物理块hash值的方法

现在，我们来细看代码中对prefill阶段物理块hash值的计算方法，也即如下部分：

```

block = self.gpu_allocator.allocate(
    seq.hash_of_block(logical_idx),
    seq.num_hashed_tokens_of_block(logical_idx))

```

hash值的计算非常重要：当两个等待做prefill的seq拥有同样的hash值时，说明它们共享一样的prompt，这时就可以重复利用已有的KV cache。（decode阶段当然也有这种操作，但有些许不同，我们在后文细聊）。

计算hash值的脚本在Sequence类的定义下（一切尽在注释中）：

```

# vllm/vllm/sequence.py
def hash_of_block(self, logical_idx: int) -> int:
    """
    计算一个逻辑块的hash值
    （注意，这里说成逻辑块hash值或者物理块hash值都可以哈，本质上都是对文本内容的hash）
    """
    # TODO This can produce incorrect hash when block size > prompt size

    # Compute the number of tokens in the sequence
    # TODO: The current hashing function is O(L^2). We should optimize
    # this in the future.

    #
=====
    # 计算截止到序号为logical_idx的逻辑块的最后一个slots为止（包含这个逻辑块），
    # 一共可以有多少个token
    #
=====
    num_tokens = self.num_hashed_tokens_of_block(logical_idx)

```

```

#
=====
# 该逻辑块的hash值 =
# hash(tuple(截止到序号为logical_idx为止这个seq的所有token_id, lora相关的配置))
# 注：“lora相关的配置”这个参数我们可以先忽略不看
#
=====

return hash(
    (tuple(self.data.get_token_ids()[0:num_tokens]), self.lora_int_id))

def num_hashed_tokens_of_block(self, logical_idx: int):
    """
    计算截止到序号为logical_idx的逻辑块为止（包含这个逻辑块），一共有多少个token
    例如block_size = 16, logical_idx = 1,
    则截止到序号为1的逻辑块为止，一共有 1*16 + 16 = 32个逻辑块
    """
    return logical_idx * self.block_size + self.block_size

```

下面这张图帮助大家象化了解prefill阶段计算block hash的方法，图中右侧给出了每个block的hash值和对应的num_tokens：

How to calculate block hash value in prefill stage

Logical Block

logical_idx

0	A	B	C	D	hash_str = hash(A~D), num_tokens = 4
1	E	F	G	H	hash_str = hash(A~H), num_tokens = 8
2	I	J	K	L	hash_str = hash(A~L), num_tokens = 12
3	M	N			hash_str = hash(A~N), num_tokens = 16

知乎 @猛猿

知乎 @猛猿

3.3 使用evictor管理物理块分配细节

那现在，我们再看进一步看下上面代码中 `block = self.gpu_allocator.allocate()` 的实现，我们重点关注 `CachedBlockAllocator` 中 `allocate` 函数，从这里开始，请大家忘记所有关于 `UncachedBlockAllocator` 的信息！！重新开始以 prefix caching 的视角来理解物理块分配的原理！（一切尽在注释中）：

```

# vllm/core/block_manager_v1.py
class CachedBlockAllocator(BlockAllocatorBase):
    """Manages free physical token blocks for a device.

    The allocator maintains a list of free blocks and allocates a block when
    requested. When a block is freed, its reference count is decremented. If
    the reference count becomes zero, the block is added back to the free list.

    """

    def __init__(self,
                 device: Device,

```

```

        block_size: int,
        num_blocks: int,
        eviction_policy: EvictionPolicy = EvictionPolicy.LRU) -> None:
self.device = device # 设备类型 (cpu/gpu)
self.block_size = block_size # 每个block的大小
self.num_blocks = num_blocks # 该设备上留给KV cache的总blocks数量

#
=====

# 跟踪当前设备上已分配的物理块的数量
# （注意，如果一个物理块被释放，但refcount不为0，则current_num_blocks不会减少）
#
=====

self.current_num_blocks = 0

#
=====

# 用于存放正在被使用的物理块
# {该物理块（也可以说是逻辑块）的hash值: 物理块}
#
=====

self.cached_blocks: Dict[int, PhysicalTokenBlock] = {}

#
=====

# 实例化驱逐器（我们马上就在后文中讲解驱逐器相关的细节），驱逐器的大致作用如下：
# 1. 当一个物理块因为refcount = 0被释放时，它先进入evictor的free_tables中，暂时不
从存储
# 中释放。因此，
# 该设备上所有可用的物理块数量=
# 正在被使用/等待被使用的物理块数量 + evictor的free_tables中的物理块数量
#
# 2. 等到后面设备上的空间真得不够时，再按照free_tables中物理块们最后一次被使用的时
间，
# 从早到晚进行排序，优先释放掉那些较早被使用的物理块
# 这个处理规则称为LRU (Least Recently Used)
#
# （暂时不释放的原因是当下在prefix caching模式，可能后面有同样prefix的seq会用到它）
#
=====

self.evictor: Evictor = make_evictor(eviction_policy) # 默认是LRUEvictor

#
=====

# count(): from itertools import count, 用于创建伪hash值，
# 主要用在decode阶段，我们在后文中会详细解释它的作用
#
=====

self.default_hash_ctr = count()

def allocate_block(self, block_hash: int,
                    num_hashed_tokens: int) -> PhysicalTokenBlock:
    """
    为哈希值为block_hash分配一个物理块
    Args:
        block_hash: 某个逻辑块的hash值。

```

对于一个seq，计算其下序号为logical_idx的逻辑块的hash值的方法如下：

hash(tuple(截止到序号为logical_idx为止这个seq的所有token_id, lora相关的配置))

num_hashed_tokens: 对于一个seq，截止到序号为logical_idx的逻辑块为止（包含这个逻辑块），一共有多少个token

```
"""
#
=====
# 如果目前已经使用的物理块数量 == 该设备上所有的物理块数量：
# [这时如果想分配新的物理块，就只能从驱逐器中根据LRU（或同LRU下根据max_hash_tokens
准则），
# 驱逐掉一块长久不用的物理块，这样才有空间分配新的物理块]
# （我们马上在下文对驱逐器的讲解中来看驱逐策略细节）
#
=====

if self.current_num_blocks == self.num_blocks:
    #
=====

    # 返回一个被选中的驱逐块block，这个block已经从驱逐器的free_tables中移走了
    #
=====

    block = self.evictor.evict()
    #
=====

    # 设定当前block对应的block hash值，直接返回当前block
    #
=====

    block.block_hash = block_hash
    block.num_hashed_tokens = num_hashed_tokens
    return block

# =====
# 如果目前已使用的物理块数量 != 该设备上所有的物理块数量，那么就新分配一个物理块
# =====
block = PhysicalTokenBlock(device=self.device,
                            block_number=self.current_num_blocks,
                            block_size=self.block_size,
                            block_hash=block_hash,
                            num_hashed_tokens=num_hashed_tokens)

# =====
# 当前已使用的物理块数量 += 1
# =====
self.current_num_blocks += 1
return block

def allocate(self,
             block_hash: Optional[int] = None,
             num_hashed_tokens: int = 0) -> PhysicalTokenBlock:
    """
    为哈希值为block_hash分配一个物理块
    Args:
        block_hash: 某个逻辑块的hash值。
        对于一个seq，计算其下序号为logical_idx的逻辑块的hash值的方法如
下：
        hash(tuple(截止到序号为logical_idx为止这个seq的所有token_id,
                    lora相关的配置))
```

```

        num_hashed_tokens: 对于一个seq, 截止到序号为logical_idx的逻辑块为止
                           (包含这个逻辑块), 一共有多少个token

        """
        #
=====
        # 如果没有传入逻辑块hash值的话, 就用一个伪hash值
        # (如前文所说, 这是decode阶段的操作, 我们马上在后文来谈这点)
        #
=====

        if block_hash is None:
            block_hash = next(self.default_hash_ctr)

        #
=====

        # 如果hash值在evictor的free_table中的话,
        # (说明拥有这个hash值的物理块, 之前因为没人引用被释放了, 就暂时存在free_tables中,
        # 现在来的这条seq拥有同样的, prefix, 所以这个物理块又可以被使用了)
        #
=====

        if block_hash in self.evictor:
            #
=====

            # 首先要保证拥有这个hash值的物理块, 肯定不在正在被使用的物理块字典中
            #
=====

            assert block_hash not in self.cached_blocks
            #
=====

            # 现在我们可以重新使用这个物理块了, 因此我们将该物理块从free_tables中移除,
            # 返回的block就是该物理块
            #
=====

            block = self.evictor.remove(block_hash)
            #
=====

            # 必须保证此时该物理块没有引用任何逻辑块(因为你刚从驱逐器的冷宫中被释放出来)
            #
=====

            assert block.ref_count == 0
            #
=====

            # 现在, 我们的这个seq可以重新使用这个物理块了。我们把它放进“正在被使用的物理块”字典中
            #
=====

            self.cached_blocks[block_hash] = block
            #
=====

            # 该物理块引用 + 1, 并直接return该物理块
            #
=====

            block.ref_count += 1
            assert block.block_hash == block_hash
            return block

```

```

#
=====
# 如果现在hash既不在驱逐器的free_tables中，也不在“当前正在被使用物理块”字典中的话
# （说明该seq装的对应文本是之前没见过的，不满足prefix可以重复利用的场景）
#
=====

if block_hash not in self.cached_blocks:
    #
=====
    # 为这个seq，分配一个新的物理块（详情参考allocate_block函数解读）
    #
=====

    self.cached_blocks[block_hash] = self.allocate_block(
        block_hash, num_hashed_tokens)
    #
=====

    # 我们从正在使用的物理块字典（cached_blocks）中取出这个新分配的物理块，
    # 并将其ref_count += 1，最后返回它
    #
=====

    block = self.cached_blocks[block_hash]
    assert block.block_hash == block_hash
    block.ref_count += 1
    return block

```

看到这里时，你或许对整体流程有了大致了解，但还是没法清晰总结。**不要急，我们先来看Evictor（驱逐器）的代码定义，在那之后，我们将对prefill阶段的运作流程做个总结。**

3.4 evictor详解

一切尽在注释中：

```

class LRUEvictor(Evictor):
    """Evicts in a least-recently-used order using the last_accessed timestamp
    that's recorded in the PhysicalTokenBlock. If there are multiple blocks with
    the same last_accessed time, then the one with the largest num_hashed_tokens
    will be evicted. If two blocks each have the lowest last_accessed time and
    highest num_hashed_tokens value, then one will be chose arbitrarily
    """

    def __init__(self):
        self.free_table: OrderedDict[int, PhysicalTokenBlock] = OrderedDict()

    def __contains__(self, block_hash: int) -> bool:
        return block_hash in self.free_table

    def evict(self) -> PhysicalTokenBlock:

        #
=====

        # 先检查驱逐器中是否有可用的物理块
        #
=====

```

```

        if len(self.free_table) == 0:
            raise ValueError("No usable cache memory left")
        #
=====
        # 先在驱逐器的free_tables中随机挑1个备选物理块
        #
=====
        evicted_block = next(iter(self.free_table.values()))
        #
=====
        # 执行驱逐策略:
        # 找到驱逐器free tables 中last accessed time最早的那个物理块,
        # 把它驱逐掉, 因为它已经很久没用了。
        # 按理来说, free_tables中的物理块都是按时间append的, 即已经排序好了, 我们第1块即可。
        # 但是若存在多个block的last_accessed一致, 我们就先移除掉包含用于做hash的tokens最多的
的那个,
        # 因为它最占存储 (这里num_hashed_tokens就是3.2节绘制图中的num_tokens)
        #
=====
        for _, block in self.free_table.items():
            if evicted_block.last_accessed < block.last_accessed:
                break

            if evicted_block.num_hashed_tokens < block.num_hashed_tokens:
                evicted_block = block

        #
=====
        # 从驱逐器的free_table中移除这个被选中的驱逐物理块
        #
=====
        self.free_table.pop(evicted_block.block_hash)

        #
=====
        # 将被驱逐的物理块的计算状态设为False
        #
=====
        evicted_block.computed = False
        return evicted_block

def add(self, block: PhysicalTokenBlock):
    self.free_table[block.block_hash] = block

def remove(self, block_hash: int) -> PhysicalTokenBlock:
    """
    将拥有指定hash值的物理块从free_tables中移除
    """
    if block_hash not in self.free_table:
        raise ValueError(
            "Attempting to remove block that's not in the evictor")
    block: PhysicalTokenBlock = self.free_table[block_hash]
    self.free_table.pop(block_hash)
    return block

```

```

def num_blocks(self) -> int:
    return len(self.free_table)

def make_evictor(eviction_policy: EvictionPolicy) -> Evictor:
    if eviction_policy == EvictionPolicy.LRU:
        return LRUevictor()
    else:
        raise ValueError(f"Unknown cache eviction policy: {eviction_policy}")

```

我们简单总结一下驱逐器的作用：

- 当一个物理块没有任何逻辑块引用时（例如一个seq刚做完整个推理），这时它理应被释放。但是在prefix caching的前提下，我们的优化思想是：即使这个物理块当前没有用武之地，可是如果不久之后来了一个新seq，它的prefix（例如system message）和这个物理块指向的内容高度一致，那么这个物理块就可以被重复使用，以此减少存储和计算开销。
- 所以，我们设置一个驱逐器（evictor）类，它的free_tables属性将用于存放这些暂时不用的物理块。
- 此时，该设备上全部可用的物理块 = *正在被使用/等待被使用的物理块数量 + evictor的free_tables中的物理块数量*
- 在prefill阶段，当我们想创建一个物理块时，我们先算出这个物理块的hash值，然后去free_tables中看有没有可以重复利用的物理块，有则直接复用
- 如果没有可以重复利用的hash块，那这时我们先检查下这台设备剩余的空间是否够我们创建一个新物理块。如果可以，就创建新物理块。
- 如果此时没有足够的空间创建新物理块，那么我们只好从free_tables中驱除掉一个物理块，为这个新的物理块腾出空间，驱逐策略如下：
 - 先根据LRU（Least Recently Used）原则，驱逐较老的那个物理块
 - 如果找到多个最后一次使用时间相同的老物理块，那么则根据max_num_tokens原则（max_num_tokens即为3.2图例中的num_tokens），驱逐其hash值计算中涵盖tokens最多的那个物理块。
 - 如果这些老物理块的LRU和max_num_tokens还是一致的话，那就从它们中随机驱逐一个

3.5 再来端详prefix caching中的prefix

看到这里，也许有个想法一直在你脑中徘徊：“使用prefix caching，是不是就意味着两个seq的prompt必须完全一致，才可以重复利用物理块呢？”下面我们再通过两个例子，帮助大家解答这个疑惑，也更好理解“prefix”的含义。

例1:

seq0					
0	A	B	C	D	hash_str = hash(A~D), num_tokens = 4
1	E	F	G	H	hash_str = hash(A~H), num_tokens = 8
2	I	J	K	L	hash_str = hash(A~L), num_tokens = 12
3	M	N			hash_str = hash(A~N), num_tokens = 16

☑: can reuse the existing block
 ✕: cannot reuse the existing block

seq1					
0	A	B	C	D	hash_str = hash(A~B), num_tokens = 4 ☑
1	E	F	G	H	hash_str = hash(A~H), num_tokens = 8 ☑
2	I	J	K	L	hash_str = hash(A~L), num_tokens = 12 ☑
3	M	X0			hash_str = hash(A~X0), num_tokens = 16 ✕

知乎 @猛猿

假设seq0现在做完了prefill，产出蓝色的4块物理块。现在进来一个seq1，我们想知道：seq1到底该怎么复用seq0的物理块？

我们根据3.1 ~ 3.3的代码流程走一遍：

- 当seq1刚进来时，我们先算好了它的逻辑块。现在要给每个逻辑块分配物理块。
- 对每个逻辑块，当我们决定是否要给它分配一个新的物理块时（一个新的物理块意味着占用了新的存储空间），我们先计算这个物理块的hash值。
- 按照这个流程，我们发现seq1的block0 ~ 2都可以复用seq0的（蓝色）
- 但是hash(seq1 block3) != hash(seq0 block3)，因此我们需要为seq1 block3（红色）开辟新空间。

例2:

seq0					
0	A	B	C	D	hash_str = hash(A~D), num_tokens = 4
1	E	F	G	H	hash_str = hash(A~H), num_tokens = 8
2	I	J	K	L	hash_str = hash(A~L), num_tokens = 12
3	M	N			hash_str = hash(A~N), num_tokens = 16

seq1					
0	X0	X1	A	B	hash_str = hash(X0~B), num_tokens = 4 ✕
1	C	D	E	F	hash_str = hash(X0~F), num_tokens = 8 ✕
2	G	H	I	J	hash_str = hash(X0~J), num_tokens = 12 ✕
3	K	L	M	N	hash_str = hash(X0~N), num_tokens = 16 ✕

☑: can reuse the existing block
 ✕: cannot reuse the existing block

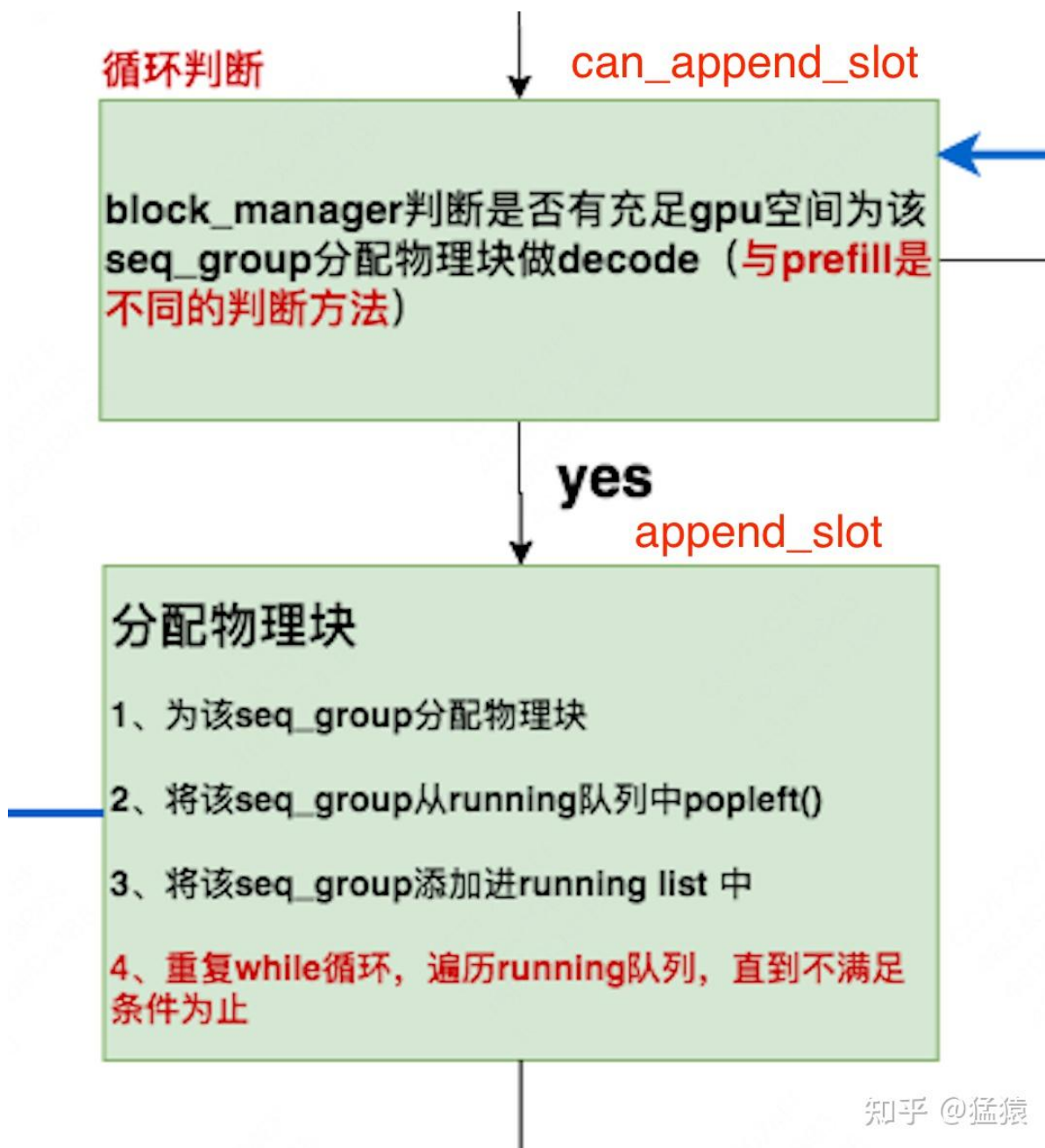
知乎 @猛猿

可以发现，尽管seq0和seq1的prompt的大部分内容是相同的，但是seq1依然不能复用seq0的prompt，这是因为KV cache的计算也需要考虑位置编码的原因。通过例1和例2，你现在是否已更好了解我们只对prefix计算hash值的原因了呢？我们再小结一下vllm中hash值计算的一些要点：

- vllm中，hash值的计算是block-level维度的
- vllm中，hash值的计算考虑了当前block及其之前所有block所维护的token值。这样做是为了找到最长可复用的prefix。

四、decode阶段的物理块分配方法

4.1 整体流程



对于每个seq_group，在上1个推理阶段，我们对它下面的每个seq都产出了1个token。所以在这个推理阶段，我们判断能否为这些seq_group分配物理块时，我们也会分成两步：

- (1) 调用 `self.block_manager.can_append_slot(seq_group)` 方法，判断是否至少能为这个seq_group下的每个seq都分配1个空闲物理块。如果可以则认为能调度这个seq_group（原因和代码分析我们在源码解读2中细讲过，这里不赘述）。
- (2) 调用 `self._append_slot(seq_group, blocks_to_copy)` 方法，实际分配物理块，这是本章节的主要内容。

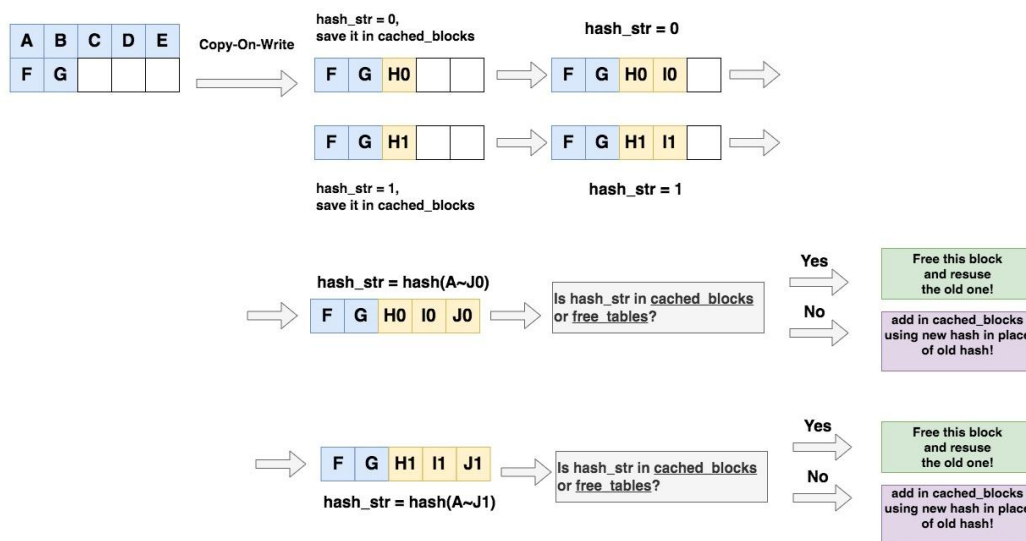
在源码解读3中，我们讲过UncachedBlockAllocator下为decode阶段分配物理块的方法（比较简单），但是现在若使用CachedBlockAllocator，考虑物理块的复用问题时，情况就更复杂一些了。所以这里我们不直接来看代码，我们先通过图解的方式，大致描述整个流程，再来看代码。

考虑下面这个使用parallel sampling做推理的例子，当n=2时，我们希望模型针对这一个prompt，产出2个推理序列结果：

```
# Parallel
Sampling("what is the meaning of life?",
SamplingParams(n=2, temperature=0.8, top_p=0.95, frequency_penalty=0.1))
```

我们来大致画一下在使用prefix caching的情况下，物理块是怎么分配的：

Prefix caching in decode phase



知乎 @猛猿

- 首先，seq正常做完prefill（蓝色部分），我们用黄色部分表示decode。
- 开始做decode。根据copy-on-write机制（参见原理篇），FG所在的block1此时被两个子序列的逻辑块引用（ref_count = 2），所以它需要被拷贝一份。这样我们就得到2个物理块，用于装H0和H1。
- 在启动copy-on-write机制的同时，我们也要重新计算物理块的hash值。和prefill阶段不同，在decode阶段，当这个物理块还没满的时候，我们会给它附一个相互不重复的默认hash值（`from itertools import count(), hash_str = next(iter(count()))`）。
- 我们会把附上hash值的物理块加入CachedBlockAllocator的cached_blocks属性中（参见3.3节代码中的讲解），我们说过，这个属性用于记录当前正在被使用的物理块。
- 两个子序列继续做decode（风平浪静的美丽日子）
- 当一个子序列用完当前物理块的所有slots时（例如当子序列1生成J0后），我们再对这个物理块重新做hash计算，计算方式是hash(A~J0)。
- 拿着这个new_hash，我们去cached_blocks（当前正在被使用的物理块列表）和free_tables（驱逐器的冷宫，曾经被使用的物理块列表）寻找。看看这两者中是否存在着相同hash值的物理块：

- 如果找到可以复用的物理块，我们就释放当前这个物理块，复用旧物理块
- 如果没有找到可以复用的物理块，我们就把当前这个物理块的旧hash值从cached_blocks中释放掉，取而代之以新hash值。

这部分vllm的代码写得比较绕，但是核心思想便是上述内容，细节我们参考接下来的代码。**在这里我们**也需关注一点：**在decode阶段，有意义的hash值其实是block-level维度的，也即只有当一个物理块满的时候，我们才对所有的prefix计算一次hash值。**所以decode阶段的prefix caching不是一种频繁地复用，而是一种累积到一定范围尽可能地长段复用，这也更加方便做KV cache管理。

4.2 append_slots入口函数

对整体流程有了把控，现在我们可以来看代码细节了，我们先看4.1（2）中说的调用入口函数，它被定义在调度器中。

调用入口（一切尽在注释中）：

```
# vllm/core/scheduler.py
def _append_slot(
    self,
    seq_group: SequenceGroup,
    blocks_to_copy: Dict[int, List[int]], # {旧物理块id: [由旧物理块copy-on-
write而来的新物理块id]}
) -> None:
    #
    =====
    # 遍历这个seq_group中状态为running的所有seq
    #
    =====
    for seq in seq_group.get_seqs(status=SequenceStatus.RUNNING):
        #
        =====
        # 为这个seq分配物理块，代码细节见下
        # ret = None时，说明可以继续使用物理块的空槽位，不需要新分配物理块
        # ret部位空时的结果为：（last_block.block_number,
new_block.block_number）
        # 前者表示源物理块，后者表示copy-on-write而来的物理块
        # （有疑惑不要紧，下文我们马上来看代码细节）
        #
        =====
        ret = self.block_manager.append_slot(seq)
        #
        =====
        # ret非None，说明采用了copy-on-write机制（参见原理篇讲解）
        # 这时我们要记录copy-on-write相关的映射关系
        #
        =====
        if ret is not None:
            src_block, dst_block = ret
            # {旧物理块id: [由旧物理块copy-on-write而来的新物理块id]}
            if src_block in blocks_to_copy:
                blocks_to_copy[src_block].append(dst_block)
            else:
```

```
blocks_to_copy[src_block] = [dst_block]
```

我们再来看 `self.block_manager.append_slot(seq)` 细节（一切尽在注释中）：

```
# vllm/core/block_manager_v1.py
def append_slot(
    self,
    seq: Sequence,
) -> Optional[Tuple[int, int]]:
    """
    为这个seq中的新token分配一个物理槽位
    """

    # =====
    # 读取这个seq的逻辑块，List[LogicalTokenBlock]
    # =====
    logical_blocks = seq.logical_token_blocks
    # =====
    # 读取这个seq的物理块，List[PhysicalTokenBlock]
    # =====
    block_table = self.block_tables[seq.seq_id]

    # =====
    # 如果物理块数量 < 逻辑块数量（说明此时需要分配新的物理块了）
    # =====
    if len(block_table) < len(logical_blocks):
        # =====
        # 需要声明物理块只允许比逻辑块少1块
        # =====
        assert len(block_table) == len(logical_blocks) - 1

        # =====
        # 如果使用滑动窗口时的逻辑（暂时不看）：
        # =====
        if (self.block_sliding_window
            and len(block_table) >= self.block_sliding_window):
            # reuse a block
            block_table.append(block_table[len(block_table) %
                                           self.block_sliding_window])

        # =====
        # 其余情况，直接分配一个新的物理块给当前序列
        # 根据是否使用prefix caching，分配方式有所不同
        # =====
        else:
            # The sequence has a new logical block.
            # Allocate a new physical block.
            new_block = self._allocate_last_physical_block(seq)
            block_table.append(new_block)
            return None

    # =====
    # We want to append the token to the last physical block.
    # 如果物理块数量=逻辑块数量：
    # =====
    last_block = block_table[-1] # 取出最后一个物理块
    assert last_block.device == Device.GPU # 声明必须是gpu物理块
```

块，

```
# =====
# 如果最后一个物理块的引用数量为1（只有1个逻辑块引用它）
# =====
if last_block.ref_count == 1:
    # =====
    # 如果你是在做prefix caching
    # =====
    if self.enable_caching:
        # =====
        # 我们可以检查下当前物理块slots是否被填满：
        # - 没有填满，不需要做任何操作
        # - 填满时，我们对它计算new_hash值，并根据new_hash寻找是否有可复用的物理块，

        # 若有，则释放当前物理块，取而代之以旧物理块
        # =====
        maybe_new_block = self._maybe_promote_last_block(
            seq, last_block)
        block_table[-1] = maybe_new_block

        # =====
        # 其余情况，我们不需要特别处理最后一个物理块，因此返回None
        # =====
        return None

# =====
# 如果最后一个物理块的引用数量为 > 1（有别的逻辑块在引用它）
# =====
else:
    # =====
    # 采用copy-on-write机制，分配一个新的物理块
    # =====
    new_block = self._allocate_last_physical_block(seq)
    block_table[-1] = new_block
    # 从该seq的block_table中释放掉旧的物理块
    self.gpu_allocator.free(last_block)
    return last_block.block_number, new_block.block_number
```

接下来我们主要看两个函数（这两点在4.1中的流程图中都有体现，大家可以对照着看）：

- `_allocate_last_physical_block`：在使用prefix caching前提下，当我们可能需要添加一个新物理块时，我们该怎么做？
- `_maybe_promote_last_block`：在使用prefix caching的前提下，如果当前物理块的slots满了，我们应该怎么做？

4.3 如何添加一个新物理块

可以对照4.1的流程图进行阅读，一切尽在注释中：

```
def _allocate_last_physical_block(
    self,
    seq: Sequence,
) -> PhysicalTokenBlock:
    """
```

```

我们在想添加新的物理块之前，调用这个函数，来判断有没有必要添加新物理块，
以及根据何种方式添加物理块
"""

# =====
# 如果不使用prefix caching，就直接分配物理块
# =====
if not self.enable_caching:
    return self.gpu_allocator.allocate()

# =====
# 使用prefix caching
# =====
# 当block_hash值为None时，在后面进入物理块allocate环节时，就会使用
next(iter(count()))
# 给一个默认值
block_hash: Optional[int] = None
# 如果该seq的最后一块逻辑块是满的
if (self._is_last_block_full(seq)):
    # 那么就截止到最后一块逻辑块（包括最后一块逻辑为止）计算所有token的block hash
    # （除了prefill阶段外，decode阶段只有在逻辑块满的时候才重新计算整串tokens的hash
    值，

    # 否则就用默认hash值）
    block_hash = seq.hash_of_block(len(seq.logical_token_blocks) - 1)
    # 如果最后一块逻辑块没有满，那么这时blockhash = None

num_hashed_tokens = seq.num_hashed_tokens_of_block(
    len(seq.logical_token_blocks) - 1)

# =====
# 使用prefix caching下的allocate方法，根据物理块的hash值，检查当前
# 是否有可复用的物理块，以此决定是新分配一个物理块，还是复用旧物理块。
# (allocate的详细注释参见本文3.3)
# =====
new_block = self.gpu_allocator.allocate(block_hash, num_hashed_tokens)

# =====
# 物理块的slots没满的情况下，我们必须确保它的ref_count = 1
# =====
if block_hash is None:
    assert new_block.ref_count == 1
return new_block

```

4.4 物理块的slots满时要如何处理

一切尽在注释中~

```

def _maybe_promote_last_block(
    self,
    seq: Sequence,
    last_block: PhysicalTokenBlock,
) -> PhysicalTokenBlock:
    # =====

```

```

# 检查当前这最后一个逻辑块是否满了，如果是，则检查是否有可复用物理块
# =====
if self._is_last_block_full(seq):
    return self._promote_last_block(seq, last_block)
else:
    return last_block

    def _promote_last_block(
self,
seq: Sequence,
last_block: PhysicalTokenBlock,
) -> PhysicalTokenBlock:
    assert self.enable_caching

#
# =====
# 如果当前最后一个逻辑块已经是满了，那么就根据所有tokens计算一个new_hash值
#
# =====
new_hash = seq.hash_of_block(len(seq.logical_token_blocks) - 1)

#
# =====
# 如果根据new_hash，这最后一个物理块在cached_tables(有人正在和你用一样的)
# 或者在驱逐器中了（驱逐器的冷宫里有和你一样的）
#
# =====
if self.gpu_allocator.contains_block(new_hash):
    #
    # =====
    # 释放最后一个物理块
    #
    # =====
    self.gpu_allocator.free(last_block)
    #
    # =====
    # 用new_hash重新分配（即复用）物理块
    #
    # =====
    return self.gpu_allocator.allocate(new_hash)
#
# =====
# 如果根据new_hash，这最后一个物理块既不在cached_tables也不在驱逐器中
# （说明只有它的默认hash值在cached_tables中）
#
# =====
else:
    # 对于这个物理块，在cached_tables中用new_hash代替old_hash（默认hash）
    self.gpu_allocator.update_hash(new_hash, last_block)
    return last_block

```